Dynamic _Cast in C++

Last Updated : 23 Jul, 2025

In C++, **dynamic_cast** is a cast operator that converts data from one type to another type at runtime. It is mainly used in inherited class hierarchies for safely casting the base class pointer or reference to derived class (called **downcasting**). To work with **dynamic_cast,** there must be one virtual function in the base class.

Syntax

```
dynamic_cast <new_type>(expression)
```

If the object being pointed to is of the correct type (or a type convertible to it), the cast succeeds. If the cast is invalid, the pointer is set to **nullptr** or it is a reference, then it throws **std::bad_cast** exception.

> ***Note:*** *A Dynamic_cast has runtime overhead because it checks object types at run time using "Run-Time Type Information". If there is a surety we will never cast to wrong object then always avoid dynamic_cast and use static_cast.*

Examples of Dynamic Cast

The below examples demonstrate how to use the dynamic cast in C++:

Downcasting using dynamic_cast

```
#include <iostream>
using namespace std;

// Base Class declaration
class Base {
public:
    virtual void print() {
```

```
        cout << "Base" << endl;
    }
};

// Derived1 class declaration
class Derived1 : public Base {
public:
    void print() {
        cout << "Derived1" << endl;
    }
};

int main() {
    Derived1 d1;

    // Base class pointer holding
    // Derived1 Class object
    Base* bp = &d1;

    // Dynamic_casting
    Derived1* dp2 = dynamic_cast<Derived1*>(bp);
    if (dp2 == nullptr)
        cout << "Casting Failed" << endl;
    else
        cout << "Casting Successful" << endl;

    return 0;
}
```

Output

```
Casting Successful
```

In this program, there is one base class and two derived classes (Derived1, Derived2), here the base class pointer hold derived class 1 object (d1). At the time of **dynamic_casting** base class, the pointer held the Derived1 object and assigning it to derived class 1, assigned valid **dynamic_casting**.

Downcasting a Non-Polymorphic Type

As we mention above for dynamic casting there must be one Virtual function. Suppose If we do not use the virtual function, then what is the result? In that case, it generates an error message "Source type is not polymorphic".

```cpp
#include <iostream>
using namespace std;

// Non-polymorphic base class
class Base {
public:
    void print() {
        cout << "Base" << endl;
    }
};

class Derived1 : public Base {
public:
    void print() {
        cout << "Derived1" << endl;
    }
};

int main() {
    Derived1 d1;
    Base* bp = &d1;

    // Dynamic_casting
    Derived1* dp2 = dynamic_cast<Derived1*>(bp);
    if (dp2 == nullptr)
        cout << "Casting Failed" << endl;
    else
        cout << "Casting Successful" << endl;

    return 0;
}
```

Output

```
main.cpp: In function ‘int main()’:
main.cpp:24:21: error: cannot ‘dynamic_cast’ ‘bp’ (of type
```

```
'class Base*') to type 'class Derived1*' (source type is
not polymorphic)
   24 |     Derived1* dp2 = dynamic_cast<Derived1*>(bp);
      |                     ^~~~~~~~~~~~~~~~~~~~~~~~~~~
```


virtual function in the base class. So, this code generates an error.

Trying Invalid Downcasting

Now, If the cast fails and new_type is a pointer type, it returns a null pointer of that type.

```
#include <iostream>
using namespace std;

class Base {
    virtual void print() {
        cout << "Base" << endl;
    }
};

// Derived1 class declaration
class Derived1 : public Base {
    void print() {
        cout << "Derived1" << endl;
    }
};

// Derived2 class declaration
class Derived2 : public Base {
    void print() {
        cout << "Derived2" << endl;
    }
};

int main() {
    Derived1 d1;
    Base* bp = &d1;

    // Dynamic Casting
    Derived2* dp2 = dynamic_cast<Derived2*>(bp);
```

```
    if (dp2 == nullptr)
        cout << "Casting Failed" << endl;
    else
        cout << "Casting Successful" << endl;

    return 0;
}
```

Output

```
Casting Failed
```

Explanation: In this program, at the time of dynamic_casting base class pointer holding the Derived1 object and assigning it to derived class 2, which is not valid dynamic_casting. So, it returns a null pointer of that type in the result.

Downcasting a Reference

Now take one more case of dynamic_cast. If the cast fails and new_type is a reference type, it throws an exception that matches a handler of type std::bad_cast and gives a warning: dynamic_cast of "Derived d1" to "class Derived2&" can never succeed.

```
// C++ Program demonstrate if the cast
// fails and new_type is a reference
// type it throws an exception
#include <exception>
#include <iostream>
using namespace std;

class Base {
    virtual void print() {
        cout << "Base" << endl;
    }
};

class Derived1 : public Base {
    void print() {
        cout << "Derived1" << endl;
    }
};

class Derived2 : public Base {
    void print() {
        cout << "Derived2" << endl;
    }
```

```
};

int main() {
    Derived1 d1;
    Base* bp = dynamic_cast<Base*>(&d1);

    // Type casting
    Derived1* dp2 = dynamic_cast<Derived1*>(bp);

    // Exception handling block
    try {
        Derived2& r1 = dynamic_cast<Derived2&>(d1);
    }
    catch (std::exception& e) {
        cout << e.what() << endl;
    }

    return 0;
}
```

Output

```
main.cpp:31:15: warning: unused variable ‘dp2’ [-Wunused-
variable]
   31 |     Derived1* dp2 = dynamic_cast<Derived1*>(bp);
      |               ^~~
std::bad_cast
```


Article Tags : C++ cpp-data-types CPP-Basics

Explore

C++ Basics

Core Concepts

OOP in C++

Standard Template Library(STL)